

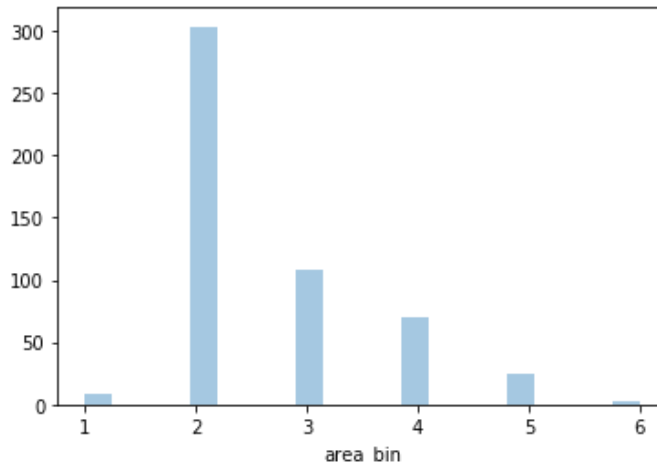
Lab 2

by Chuan Sun

I tried decision tree and K nearest neighbor algorithm on the data set to classify the area target. Some preprocessing and feature engineering processes are done beforehand:

- 1) apply label encoding on the "month" and "day" column
- 2) take the logarithm of area column
- 3) cut the logged area values into 6 bins for classification

It could be observed that there's an obvious class bias and some minority classes exist:



So I did some oversampling using SMOTE mechanism, which tends to add more sample points on the line between two minority class points. There are 1810 samples after oversampling.

Based on that, I applied decision tree and KNN on both original and oversampled dataset. Besides, I also self-implemented 2 functions to calculate sensitivity and specificity matrix for multi-label classification.

1. Decision Tree

I started with a simple decision tree on the original dataset (without oversampling), with gini criterion and max_depth to be 3. It reached an accuracy of .61.

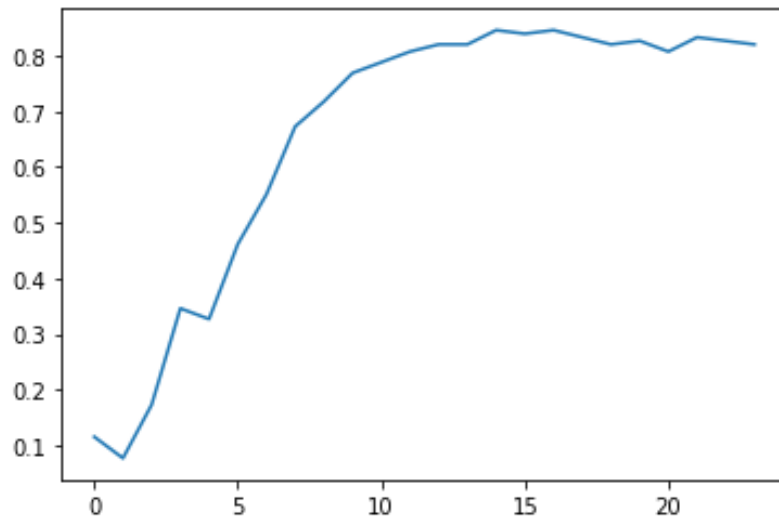
Accuracy: 0.6089743589743589						Accuracy: 0.17307692307692307					
[[0 3 0 0 0 0]						[[3 0 0 0 0 0]					
[0 95 0 0 0 0]						[47 16 0 19 10 3]					
[0 28 0 3 0 0]						[13 5 0 11 1 1]					
[0 19 0 0 0 0]						[10 2 0 6 0 1]					
[0 7 0 0 0 0]						[4 0 0 1 1 1]					
[0 1 0 0 0 0]]						[0 0 0 0 0 1]]					
sensitivity			specificity			sensitivity			specificity		
label			label			label			label		
1.0	0.0	1.000000	1.0	1.000000	0.244898	1.0	1.000000	0.244898	1.0	1.000000	0.244898
2.0	1.0	0.000000	2.0	0.168421	0.611111	2.0	0.168421	0.611111	2.0	0.168421	0.611111
3.0	0.0	1.000000	3.0	0.000000	1.000000	3.0	0.000000	1.000000	3.0	0.000000	1.000000
4.0	0.0	0.969388	4.0	0.315789	0.403846	4.0	0.315789	0.403846	4.0	0.315789	0.403846
5.0	0.0	1.000000	5.0	0.142857	0.702703	5.0	0.142857	0.702703	5.0	0.142857	0.702703
6.0	0.0	1.000000	6.0	1.000000	0.812500	6.0	1.000000	0.812500	6.0	1.000000	0.812500

original

oversampled

Observing from the confusion matrix, it seems like most classes other than 2 are misclassified, with sensitivities of 0. Also, it is a surprise that the accuracy on oversampled dataset was so low. Thus I proceed with parameter tuning.

I started off with an important parameter: `max_depth` which controls the depth of the tree. Accuracy apparently improves with more depths:



The performance peaks at around 14, and slightly fluctuates afterwards. So `max_depth` to be 14 would be a good way to avoid overfitting.

In order to try tuning more parameters more efficiently, I self-implemented a grid search that outputs the best model and accuracy based on any given parameter space.

```
1680 combinations in total. Metric: accuracy
1680 / 1680, best comb: {'criterion': 'gini', 'splitter': 'best', 'max_depth': None, 'max_features': 5, 'min_samples_split': 2, 'min_samples_leaf': 1}, best score: 0.7974217311233885
best params: {'criterion': 'gini', 'splitter': 'best', 'max_depth': None, 'max_features': 5, 'min_samples_split': 2, 'min_samples_leaf': 1}
max accuracy: 0.7974217311233885
```

The grid search outputs that, the best parameters on the oversampled dataset would be:

- criterion: gini
- splitter: best
- max_depth: None, which means no limit on the max depth
- max_features: 5
- min_samples_split: 2
- min_samples_leaf: 1

The best accuracy outputted was .797, very close to the one that we got from simply tuning `max_depth`. Thus we could see that, `max_depth` played a decisive role here for classification accuracy.

Going further, I'm more and more curious how boosting and bagging would help on this dataset. I tried random forest with grid search but the performance didn't improve much, but when doing boosting, there were some good news.

```
720 combinations in total. Metric: accuracy
720 / 720, best comb: {'n_estimators': 800, 'loss': 'deviance', 'learning_rate': 0.05, 'max_depth': None, 'min_samples_leaf': 1, 'max_features': 7}, best score: 0.9060773480662984
best params: {'n_estimators': 800, 'loss': 'deviance', 'learning_rate': 0.05, 'max_depth': None, 'min_samples_leaf': 1, 'max_features': 7}
max accuracy: 0.9060773480662984
```

A Gradient Boosting classifier with 800 trees, deviance loss and 7 max_features would achieve an accuracy of around .906, which is a big improvement on one single tree. Take a look at the sensitivities and specificities:

```

Accuracy: 0.8747697974217311
[[ 96  1  0  0  0  0]
 [ 2 57 24 11  3  0]
 [ 1 10 61  5  1  0]
 [ 0  3  5 76  2  0]
 [ 0  0  0  0 106  0]
 [ 0  0  0  0  0 79]]
      sensitivity  specificity
label
1.0      0.989691      0.992147
2.0      0.587629      0.967593
3.0      0.782051      0.934537
4.0      0.883721      0.961446
5.0      1.000000      0.984000
6.0      1.000000      1.000000

```

Wow! We can observe that there's a huge improvement on sensitivities of minority classes, thanks to the SMOTE oversampling. While on the other hand, accuracy on class 2 declined a little bit, which would be the direction of further performance tuning.

2. K Nearest Neighbor

Frankly speaking, I love the idea of KNN for its straightforwardness and interpretability. I'm also curious about KNN's performance on oversampling. Let's proceed.

I started with a simple KNN on the original dataset with `n_neighbors` set to 5 (by default). It reached .50 accuracy, a little bit lower than the first untuned decision tree. Specificities are good, but sensitivities on minority suffers.

```

Accuracy: 0.5
[[ 0  2  1  0  0  0]
 [ 3 73 18  1  0  0]
 [ 1 25  5  0  0  0]
 [ 0 17  2  0  0  0]
 [ 0  5  2  0  0  0]
 [ 0  1  0  0  0  0]]
      sensitivity  specificity
label
1.0      0.000000      0.951220
2.0      0.768421      0.090909
3.0      0.161290      0.760417
4.0      0.000000      0.987342
5.0      0.000000      1.000000
6.0      0.000000      1.000000

```

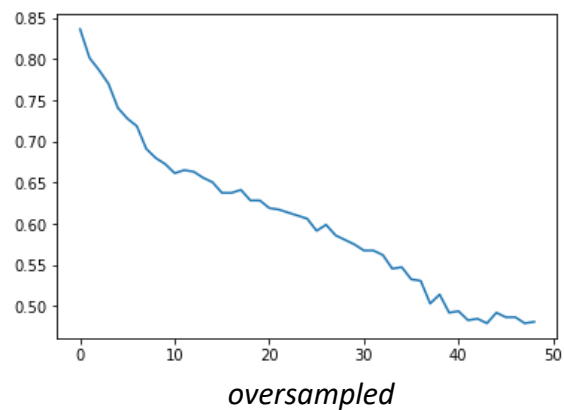
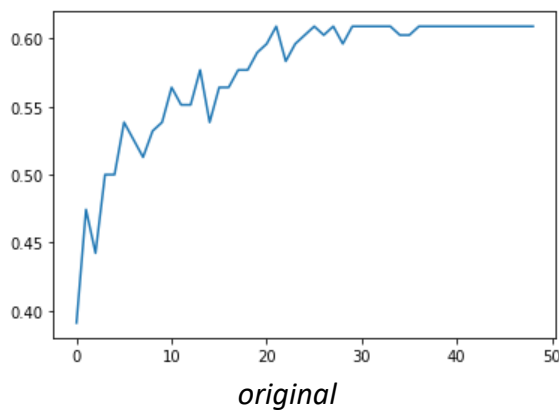
Then try our luck by applying the same KNN on the oversampled dataset:

```

Accuracy: 0.7403314917127072
[[92  0  2  0  0  3]
 [ 7 28 24 19 16  3]
 [ 4 16 42 10  0  6]
 [ 1  3  7 74  1  0]
 [ 1  4  4  5 92  0]
 [ 0  3  1  1  0 74]]
      sensitivity  specificity
label
1.0      0.948454      0.959752
2.0      0.288660      0.935000
3.0      0.538462      0.904523
4.0      0.860465      0.903581
5.0      0.867925      0.948012
6.0      0.936709      0.964706

```

It improved a lot! But TPR on class 2 declined as well. And it's time to compare different `n_neighbors`. Below are two charts depicting the performance change with `n_neighbors` on the original and oversampled dataset.



I'm kind of surprised to find a completely reversed performance trend on original dataset and oversampled dataset. For the original dataset, it's natural that increasing `n_neighbors` would take more neighbors into consideration, thus improving TPR (sensitivity) for the majority class. While caring for more neighbors simply reduces the chance of minority classes being correctly classified, thus making it difficult to reduce TNR (specificity) for minority classes, and this is why there was a performance limit.

While for the oversampled dataset, accuracy sharply declined as `n_neighbors` grew. I think the reason behind that is similar as above. A small `n_neighbors` would do well on most of the minority class points.

Accuracy: 0.8360957642725598

```
[[ 93  0  1  0  0  3]
 [  5 37 22 17 13  3]
 [  1  6 66  4  0  1]
 [  0  1  2 81  2  0]
 [  0  5  0  0 101  0]
 [  0  1  2  0  0 76]]
```

	sensitivity	specificity
label		
1.0	0.958763	0.983651
2.0	0.381443	0.969767
3.0	0.846154	0.934940
4.0	0.941860	0.946701
5.0	0.952830	0.959239
6.0	0.962025	0.981818

KNN with `n_neighbors` 1

Accuracy: 0.6721915285451197

```
[[92  1  1  0  0  3]
 [10 23 18 23 17  6]
 [  5  7 26 24  2 14]
 [  2  3  6 70  4  1]
 [  5  4  9  7 78  3]
 [  0  0  2  0  1 76]]
```

	sensitivity	specificity
label		
1.0	0.948454	0.925424
2.0	0.237113	0.957983
3.0	0.333333	0.904000
4.0	0.813953	0.845272
5.0	0.735849	0.922830
6.0	0.962025	0.914557

KNN with `n_neighbors` 10

From the above confusion matrix, it can be observed that sensitivities for both majority and minority class declines as `n_neighbors` grew from 1 to 10. Theoretically, considering more neighbors acts against the mechanism of SMOTE oversampling.

**Note: the detailed codes and notebooks for this lab can be found at:*

Decision Tree:

https://github.com/telenovelachuan/data_science/blob/master/EECS_738_machine_learning/lab2/decision_tree.ipynb

KNN:

https://github.com/telenovelachuan/data_science/blob/master/EECS_738_machine_learning/lab2/KNN.ipynb